The Octavius Protocol

CTF Writeup

Category: Forensics / Crypto / Network **Points:** 200

FLAG

`r00t{d0c_0ct4v10u5_w4s_n0t_g3n1u5}`

Challenge Description

> *Otto Octavius kept his fusion-reactor research in a private git-tracked lab notebook. After the accident, he scrubbed it - or thought he did. You have two things: an artifact and a live target.*

Entry point: `the-octavius-protocol-8950.labs.perfectroot.io`

Overview

The challenge had three stages:

1. **Git archaeology** — recover a commit Otto thought he'd scrubbed from history.
2. **Crypto break** — exploit a nonce/key-reuse bug (a "two-time pad") to recover a hidden override token.
3. **Live service exploitation** — use the recovered token to answer an HMAC challenge-response from the live reactor control service and get the flag.

Stage 1 — Recovering the Scrubbed Commit

The provided archive contained a git repo whose visible history showed only a single clean commit. Running `git fsck` to look for unreachable objects immediately revealed what Otto had tried to hide:

CRUCIAL STEP — Finding the dangling commit

```
┌──(enos㉿kali)-[~/competition/player_dist/octavius-labs]
└─$ cd player_dist
tar -xzf octavius-labs.tar.gz
cd octavius-labs
git fsck --full --unreachable --no-reflog
Checking ref database: 100% (1/1), done.
Checking object directories: 100% (256/256), done.
unreachable blob e08f18c9a67dca1dacb2cc57f90507aafd7d58d5
unreachable blob 6fbbea844d696ed77ead67fb4ff3d07985af4906
unreachable tree b7c7ff2b3bb7d6dadb1831b4077b599c40e05bb7
unreachable blob 5862aaea5164a5ae114d0bcfe54f20b1d9fd4804
unreachable blob 99787aa401565c062d0f2e6e7bbff55b266e2656
unreachable commit 1b401f0753cc60d974c567c5b3e85235ef6c8266
```

A commit never referenced by any branch/tag isn't deleted by `git reset` or history rewrites alone — the objects stay in the object store until garbage collected. That dangling commit, `1b401f075...`, was Otto's "pre-incident backup."

Extracting the blobs it pointed to:

```
git show 1b401f0753cc60d974c567c5b3e85235ef6c8266
mkdir ../extracted
git cat-file -p e08f18c9a67dca1dacb2cc57f90507aafd7d58d5 > ../extracted/calibration_checker
git cat-file -p 5862aaea5164a5ae114d0bcfe54f20b1d9fd4804 > ../extracted/calibration_log.enc
git cat-file -p 99787aa401565c062d0f2e6e7bbff55b266e2656 > ../extracted/override.enc
git cat-file -p 6fbbea844d696ed77ead67fb4ff3d07985af4906 > ../extracted/notes.md
chmod +x ../extracted/calibration_checker
```

This produced four artifacts:

- `calibration_checker` — an ELF binary

- `calibration_log.enc` — 64 bytes of "encrypted" calibration data
- `override.enc` — 64 bytes of "encrypted" override data
- `notes.md` — Otto's lab journal entry

The journal entry was the key hint:

CRUCIAL STEP — The confession that gives away the crypto bug

```
┌──(enos㉿kali)-[~/competition/player_dist/octavius-labs]
└─$ cat ../extracted/notes.md
# Lab Journal — O. Octavius

Day 212.

Rebuilt the arm calibration logger again. That wall-crawler keeps
finding my config dumps in plaintext, so from now on every calibration
log for a session gets encrypted before it touches disk. One key, one
counter, every time — I'm not made of CPU cycles to burn re-deriving a
fresh one for every single log line.

Arm III's override code lives in this session's log too. If anyone
ever gets this far, they've already seen Arm III's own self-test
output somewhere — they just have to notice it.

— O.
```

"One key, one counter, every time" is a direct confession of a **nonce/keystream reuse** vulnerability — a stream-cipher cardinal sin.

Stage 2 — Breaking the Crypto (Two-Time Pad)

Inspecting the two `.enc` files byte-for-byte showed they shared an identical first 16 bytes — the nonce:

```
5e b0 db a1 df 40 f2 e9 7c 18 8f d4 d9 ff 13 82   <- calibration_log.enc, first 16 bytes
5e b0 db a1 df 40 f2 e9 7c 18 8f d4 d9 ff 13 82   <- override.enc, first 16 bytes (identical)
```

Same nonce + same key + same counter means the remaining 48 bytes of each file were XORed with the **exact same keystream**. That's a classic two-time pad: XOR the two ciphertext bodies together and the keystream cancels, leaving `plaintext1 XOR plaintext2`. Recover either plaintext and you recover the other.

The `calibration_checker` binary supplied the missing known-plaintext. Running it with no / bad args revealed usage text, and probing further surfaced a hidden `--selftest` flag:

CRUCIAL STEP — Finding the known-plaintext crib

```
┌──(enos㉿kali)-[~/competition/player_dist/extracted]
└─$ cd ../extracted
./calibration_checker --selftest
ARM-03 CALIBRATION OK TORQUE=0417 TS=1700000000
```

That self-test string is exactly the plaintext Otto's journal hinted at ("they've already seen Arm III's own self-test output somewhere"). With `calibration_log.enc`'s plaintext known, the keystream — and then `override.enc`'s plaintext — can both be recovered:

CRUCIAL STEP — Two-time pad XOR attack → recovers the override token

```
┌──(enos㉿kali)-[~/competition/player_dist/extracted]
└─$ python3 << 'EOF'
with open('calibration_log.enc','rb') as f: c1 = f.read()
with open('override.enc','rb') as f: c2 = f.read()

body1, body2 = c1[16:], c2[16:]
known_pt = b'ARM-03 CALIBRATION OK TORQUE=0417 TS=1700000000\n'

ks = bytes(a ^ b for a, b in zip(body1, known_pt))
override_pt = bytes(a ^ b for a, b in zip(body2, ks))
print(override_pt)
EOF
b'OVERRIDE-TOKEN:601a1160a7497d0a912b94bf92ce03a3\n'
```

Recovered override token: `601a1160a7497d0a912b94bf92ce03a3`

Stage 3 — The Live Target

Finding the service

The entry point resolved to an internal lab IP (`10.200.0.2`) reachable only once connected to the lab network/VPN. Plain HTTP/HTTPS on 80/443 refused connections — the actual service lived on the port named in the challenge card itself, `8950`:

```
┌──(enos㉿kali)-[~/competition/player_dist/extracted]
└─$ curl -v http://the-octavius-protocol-8950.labs.perfectroot.io:8950/
*   Trying 10.200.0.2:8950...
* Host the-octavius-protocol-8950.labs.perfectroot.io:8950 was resolved.
* IPv6: (none)
* IPv4: 10.200.0.2
* Established connection to the-octavius-protocol-8950.labs.perfectroot.io (10.200.0.2 port 8950) from 10.8.0.191 port 55162
* using HTTP/1.x
> GET / HTTP/1.1
> Host: the-octavius-protocol-8950.labs.perfectroot.io:8950
> User-Agent: curl/8.22.0
> Accept: */*
>
* Request completely sent off
* Received HTTP/0.9 when not allowed
< OCTAVIUS REACTOR CONTROL
* Invalid response header
* closing connection #0
curl: (1) Received HTTP/0.9 when not allowed
```

curl's HTTP parser choked on the banner — this isn't an HTTP service at all, it's a raw TCP protocol. Switching to `nc` confirmed it:

```
┌──(enos㉿kali)-[~/competition/player_dist/extracted]
└─$ nc the-octavius-protocol-8950.labs.perfectroot.io 8950
OCTAVIUS REACTOR CONTROL
CHALLENGE 22b710b85144545b5f4afbd65ea29970
```

Discovering the protocol

Sending the raw token wasn't accepted:

```
┌──(enos㉿kali)-[~/competition/player_dist/extracted]
└─$ nc the-octavius-protocol-8950.labs.perfectroot.io 8950
OCTAVIUS REACTOR CONTROL
CHALLENGE 0ab70a5951ecf211228e2c3aeac6a3c7
601a1160a7497d0a912b94bf92ce03a3
ERR unknown command
```

The service expected a structured command. Asking for `HELP` revealed the protocol:

CRUCIAL STEP — Learning the auth protocol

```
┌──(enos㉿kali)-[~/competition/player_dist/extracted]
└─$ nc the-octavius-protocol-8950.labs.perfectroot.io 8950
OCTAVIUS REACTOR CONTROL
CHALLENGE 423365d50006d36c67718f31da88a75f
HELP
Octavius Reactor Control
Commands:
  HELP                this text
  VERSION              build info
  OVERRIDE <hmac_hex>    submit the signed override response to the current challenge
  QUIT
```

So the server issues a random challenge nonce per connection, and expects `OVERRIDE <hmac_hex>` — an HMAC of that challenge, signed with the recovered override token as the key.

Solving the challenge-response

Since the challenge nonce is per-connection, the HMAC has to be computed live within the same TCP session. A small

Python script handled the full exchange (connect → read challenge → HMAC-SHA256 with the token → submit `OVERRIDE`):

```
import socket, hmac, hashlib

HOST = "the-octavius-protocol-8950.labs.perfectroot.io"
PORT = 8950
TOKEN = b"601a1160a7497d0a912b94bf92ce03a3"

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((HOST, PORT))

def recv_line():
    data = b""
    while not data.endswith(b"\n"):
        chunk = s.recv(1)
        if not chunk:
            break
        data += chunk
    return data.decode(errors="replace").strip()

banner = recv_line()
print("BANNER:", banner)

challenge_line = recv_line()
print("CHALLENGE LINE:", challenge_line)
challenge = challenge_line.split()[1]
print("CHALLENGE:", challenge)

digest = hmac.new(TOKEN, challenge.encode(), hashlib.sha256).hexdigest()
print("HMAC-SHA256:", digest)

cmd = f"OVERRIDE {digest}\n"
s.sendall(cmd.encode())

resp = s.recv(4096)
print("RESPONSE:", resp.decode(errors="replace"))

s.close()
```

Running it against the live service produced the flag on the first attempt — HMAC-SHA256 was the right algorithm:

CRUCIAL STEP — Solving the HMAC challenge and capturing the flag

```
 python3 solve.py
BANNER: OCTAVIUS REACTOR CONTROL
CHALLENGE LINE: CHALLENGE bb8c6f211a8acc7898b9abf1d92aeee7
CHALLENGE: bb8c6f211a8acc7898b9abf1d92aeee7
HMAC-SHA256: 0986b749faa606037503f1a4e013017887316a7d10bf1782aebcbeb14d94d64d
RESPONSE: OVERRIDE ACCEPTED. FLAG r00t{d0c_0ct4v10u5_w4s_n0t_g3n1u5}
```

Flag

r00t{d0c_0ct4v10u5_w4s_n0t_g3n1u5}

Summary of the Vulnerability Chain

Stage	Weakness	Technique
1	"Deleted" commit still present as a dangling git object	`git fsck --unreachable` + `git cat-file`
2	Stream cipher nonce/key/counter reuse	Two-time pad: XOR two ciphertexts sharing a keystream, recover plaintext using a known-plaintext crib (`--selftest` output)
3	Challenge-response auth, correct algorithm had to be inferred	HMAC-SHA256(key=recovered token, msg=server challenge)

Each stage fed directly into the next: the git history leak exposed the crypto bug *and* the oracle (`--selftest`) needed to exploit it, and the crypto break produced the credential needed to authenticate against the live service.